

SudokuCLI

本次实验项目的实验内容为“实现基于命令行的数独游戏应用”。

其基本功能应该包括：

- 生成新的数独游戏
- 显示数独
- 允许用户在某个格子中放入数字
- 检查数独合法性
- ...

且操作必须使用命令行完成。

根据上述要求进行考虑，我们最终决定以 **REPL** 形式对该数独应用进行开发。

程序设计

本部份将挑选几个程序设计中的亮点部份进行说明。虽然这一部份可能更加倾向于程序开发中的细节，但其仍然于用户交互体验有着密不可分的联系。

REPL 交互设计

REPL(Read-Eval-Print Loop)，是一种命令行交互的模式。相比于传统的通过命令行+参数来出发特定程序行为的形式，**REPL** 可以让程序持续在前台运行，并且在内存中保存内容，为用户提供更好的交互体验。

当我们使用传统命令行程序触发格式时，一种可能的交互示例如下：

```
system> sudoku create
...
system> sudoku put x x x
...
```

当我们使用 **REPL 形式** 开发交互式 CLI 时，一种可能的交互如下：

```
system> sudoku

SudokuCLI> create
...
SudokuCLI> put x x x
...
SudokuCLI> ...
SudokuCLI> quit
```

```
system>
```

经过对比可以发现，后者在操作上更加便捷和符合直觉。

关于 REPL 交互式 CLI 的实现细节，参见：“功能实现 - REPL”

高阶 REPL 功能设计

详见附件“Advance REPL Design”

富文本显示

在文本内容显示上，本项目使用了 `rich` 显示库提供的 ANSI Escape Sequence 显示能力，使在命令行显示彩色字符成为可能。

Markdown 文档支持

一方面，项目在显示文档内容时，会使用 Markdown 格式对文档进行编写。编写完成的合法 Markdown 文档可以直接通过 `rich` 进行格式化输入，相对于纯文本大大提高了用户的阅读体验。

```
from rich import Markdown
```

```
md_str: str
cmd.poutput(Markdown(md_str))
```

在上述代码示例中，`md_str` 必须是一个格式正确的 Markdown 文档字符串。

Markup 标记支持

本程序支持对于简易 Markup 标记后的字符串进行打印：

```
cmd.poutput('[red bold]Bold text with red color![/red bold]')
```

关于富文本显示的详细实现，详见“功能实现 - 富文本显示”

参数提取支持

本程序在参数设计和提取上，向现有的通用规范进行看齐，并通过合理设计 `argparser` 解码器，为用户提供良好的命令行传参体验。

这里，我们使用 `newgame` 指令的参数设计作为例子。

命令帮助文档

得益于对于 `argparser` 的合理设置，程序可以为用户提供包含各种帮助信息和说明的提示文档：

```
usage: newgame [-h] [-d DIFFICULTY] [-s {rotate-90,rotate-180,mirror-x,
mirror-y,mirror-xy}] [-t TEMPLATE]
```

Create a new game

optional arguments:

```
-h, --help            show this help message and exit
-d DIFFICULTY, --difficulty DIFFICULTY
                        Set the difficulty of the newly generated game.
                        Should be a float number between 0 and 1, larger number will lead to more empty blocks, thus, a more challenging game.
                        (default: None)
-s {rotate-90,rotate-180,mirror-x,mirror-y,mirror-xy}, --symmetry {rotate-90,rotate-180,mirror-x,mirror-y,mirror-xy}
                        Specify the symmetry pattern of the new game (default: None)
-t TEMPLATE, --template TEMPLATE
                        Specify the grid template of the new game (default: None)
```

自动补全

参数正确定义后，可以使用 Tab 对指令进行补全

```
Sudoku CLI > newg      # 此时按下 Tab
Sudoku CLI > newgame
Sudoku CLI > --tem     # 此时按下 Tab
Sudoku CLI > newgame --template
```

可选参数

本程序允许为指令添加可选的命名参数 (Named Arguments)。比如对于 `export` 指令中，`-r/--rowsep` 可以省略，若不提供，其默认值为空字符串。

对于更高阶的可选参数，参见 `export` 指令中的 `-f/--file` 参数。这种参数有三种状态：

```
export
export -f
export -f output.json
```

- 第一种情况，参数标志完全未出现，采用 `default` 值 (`None`)。
- 第二种情况，参数标志出现，但没有提供值，采用 `const` 值 (`game.json`)。
- 第三种情况，参数标志和值均显式指定，采用指定的值。

这种设计可以提高指令输入的效率。通过将常用的参数值设置为 `const`，可以减少用于输入的指令长度。比如用户想要到处到默认文件 `game.json` 时，只需要输入 `e -f` (`export --file`) 即可，而不需指定文件名称。

互斥参数组

部份指令可能会包含互斥的参数组。即：某一组参数中，至多只能有一组参数传入。

具体的例子为 `loadgame` 指令。该指令支持从文件读取游戏，或者直接从命令行输入游戏字符串进行导入。两者为互斥选项，用户只能选择一种方式进行导入。

互斥参数组会在帮助文档中显示。

```
usage: loadgame [-h] (-s STRING | -f [FILE])
```

Load an existing game from a data string

optional arguments:

```
-h, --help            show this help message and exit
-s STRING, --string STRING
                        The string format data of a game. You could export
                        a game to string using "export" command (default: None)
-f [FILE], --file [FILE]
                        Load game from file. Use 'game.json' if no args
                        followed (default: None)
```

同时，如果用户尝试同时传入两个互斥参数，系统会提供准确的报错信息：

```
Sudoku CLI> l -f -s test
usage: loadgame [-h] (-s STRING | -f [FILE])
Error: argument -s/--string: not allowed with argument -f/--file
```

功能实现

指令列表

```
Sudoku CLI> hh
```

Documented commands (use 'help -v' for verbose/'help <topic>' for details):

Category 1: Sudoku

```
=====
```

check	Check if there's any conflict in current game
export	Export sudoku game
home	Show the home screen of this game program
loadgame	Load an existing game from a data string
newgame	Create a new game
put	Put or update a box of the sudoku
show	Show current game
solve	Show the solution of current sudoku game
step	
step_revert	

step_show

Category 2: Documentation

```
=====
doc
```

Category 3: System

```
=====
alias          Manage aliases
cls            Clear all content on screen
help          List available commands or provide detailed help
for a specific command
history        View, run, edit, save, or clear previously entered
commands
quit           Exit this application
```

新建游戏

Sudoku CLI> n -h

usage: newgame [-h] [-d DIFFICULTY] [-s {rotate-90,rotate-180,mirror-x,mirror-y,mirror-xy}] [-t TEMPLATE]

Create a new game

optional arguments:

-h, --help show this help message and exit
-d DIFFICULTY, --difficulty DIFFICULTY
Set the difficulty of the newly generated game.
Should be a float number between
0 and 1, larger number will lead to more empty
blocks, thus, a more challenging
game. (default: None)
-s {rotate-90,rotate-180,mirror-x,mirror-y,mirror-xy}, --symmetry
{rotate-90,rotate-180,mirror-x,mirror-y,mirror-xy}
Specify the symmetry pattern of the new game (default: None)
-t TEMPLATE, --template TEMPLATE
Specify the grid template of the new game (default: None)

此外，本程序的新游戏创建支持各种定制化选项，参见附件“Newgame Customization”

Create newgame with template “cat”

Generating new game with difficulty *None*
Generate a valid game from template may takes some time, please wait for a sec...
New sudoku game generated!

Current Sudoku									
	1	2	3	4	5	6	7	8	9
1	4	7						2	6
2	9		8				5		4
3	5			8	4	7			9
4	8	6	3				2	4	5
5	2	4	5				7	9	1
6	1								8
7		5			2			1	
8			2				9		
9				9	1	3			

Original / Filled
Filled: 35/81 (43.21%)

image

Create newgame with -d 0.4 and -s rotate-90

Generating new game with difficulty *0.4*
New sudoku game generated!

Current Sudoku									
	1	2	3	4	5	6	7	8	9
1	6	4	1	7					8
2				1	3	5	4		7
3		7	3		4	8	9		2
4		2	6		7			4	5
5		5	7	4	8	6	2	9	
6	4	8			2		3	7	
7	7		5	8	6		1	2	
8	8		4	2	1	9			
9	9					7	6	8	4

Original / Filled
Filled: 49/81 (60.49%)

image

输入数据

Usage: put [-h] {1, 2, 3, 4, 5, 6, 7, 8, 9} {1, 2, 3, 4, 5, 6, 7, 8, 9}
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

Put or update a box of the sudoku

positional arguments:

{1, 2, 3, 4, 5, 6, 7, 8, 9}
Row number of the box
{1, 2, 3, 4, 5, 6, 7, 8, 9}
Column number of the box
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
Value of the box

optional arguments:

-h, --help show this help message and exit

查看游戏

游戏的查看是用户交互频率最高的功能之一。

对于此功能，我们提供的功能有：

- 标准的可变长度网格显示
- 候选数字显示
- 输入格子的高亮
- 冲突格子的高亮
- 高亮格式自定义

Sudoku CLI> sh -h

Usage: show [-h] [-c] [-p]

Show current game

optional arguments:

-h, --help show this help message and exit
-c, --candidates Show candidates of not filled position
-p, --perserve-terminal Do not clear terminal before showing the sudoku

```
Current Sudoku
  1  2  3  4  5  6  7  8  9
1  2  1  3  6  5  9  8
2  9  4  8  2  1
3  4  5  8  2  1
4  8  2  5  1
5  3  9  6  7  1
6  7  1  3  9  5  2
7  3  4  8  9  1
8  1  5  4  3  8
9  1  3  4

Original / Filled / Conflicts
Filled: 42/81 (51.85%)

Command Cheatsheet:
- put / p      Put new number in blocks.
- show / sh    Show current game.
- show -c     Show current game with candidates hint.
- solve       Show the possible solve of current game.

Usages Of Helps:
- help -v / hh Show detailed help of all commands.
- help [command] Show help of a certain command.

Enter "home" or "ho" to show home page
Enter "quit" or "q" to exit

(1, 2) <== Conflict ==> (6, 2) [Both 1]
(6, 2) <== Conflict ==> (1, 2) [Both 1]
```

image

游戏导入导出

于此同时，程序输出支持命令行重定向，允许用户将程序输出重定向到其他程序，或者储存到文件中。

详见附件“Output Redirection”。

REPL

这一部份属于开发细节。如果您只对面向用户的功能实现感兴趣，可以跳过这一部份。

本项目使用 Python 进行开发。Python 官方内置了基础的 cmd 以及 argparse 库为在 Python 中开发 CLI 程序提供了很好的支持。

本程序主要基于 `cmd2` 库对 CLI 基础逻辑进行开发。其提供了很多标准化的命令行程序支持，包括但不限于：

- 基础的内置命令
- Settable 配置项
- 指令历史管理和调出
- 指令自动补全
- ...

富文本显示

这一部份属于开发细节。如果您只对面向用户的功能实现感兴趣，可以跳过这一部份。

我们通过编写对于 `cmd2.Cmd` 的 Mixin 类 `RichCmd`，来实现 `cmd.poutput()` 以及其他函数对于富文本字符串的支持。

相关代码，参见 `tools/cmd2_rich_mixin.py`

于此同时，我们对于 `argparse` 和 `cmd2` 两个库进行了不同程度的轻度 monkey patching，来修复其部份不正常行为和 Bug。同时将 `cmd2.poutput()` 和其他输出函数的实现进行替换。

详见 [此 GitHub Issue](#)

Advanced REPL Design

Our system is developed based on standard REPL conventions, thus there's bunch of useful shortcuts available out of the box.

Command History

You could use `history` command to list or manage your command history. Also, **you could use `↑` and `↓` on keyboard to quickly navigated to previous executed commands.**

Tab Auto-complete

Theoretically, all places that is allowed to input command will be able to use Tab auto-complete feature. Program will **check and show all possible following part of the command** based on the part that has already be input. Also if **there is only one possible input left, Tab will auto-complete that command** for you.

Here is an example:

```
(Cmd) s      # press tab will print:
# sh         show      so        solve
(Cmd) sol    # press tab will trigger auto-complete
(Cmd) solve
```

Command Alias

Our program support user-defined command alias (We've added some common alias by using startup scripts, like p for put, n for newgames etc. You could run `alias list` to check all alias currently available in the program)

Also, you could prefer to add your own alias using `alias` command. Check out more info using `help alias`.

Alias will also works with paramters, for example, if we have n as the alias for newgame, then both of following command is a legal command:

```
newgame -d 0.5
n -d 0.5
```

Startup Scripts

You could edit `./sodukurc` files to add your own startup script, which will be executed upon CLI startup.

Newgame Customization

This game provide several features that allows you to comtomize the shapes of the game generated when using `newgame` command.

Difficulty

You could use `-d/--difficulty` to specify the factor of the difficulty of the newly created game. Difficulty should be a float number in range `[0, 1]`.

```
newgame -d 0.5
```

Generally, a **higher number represents more blank grids, thus a more challenging game**.

Symmetry

When generating games by difficulty, you could also specify the symmetric behaviour! All supported symmetry pattern is listed below:

- rotate-90
- rotate-180
- mirror-x

- mirror-y
- mirror-xy

You could use this args with newgame command:

```
newgame -d 0.5 --symmetry rotate-90
```

Templates

Game Template is a feature in this program that allow you to **completely take control of the shape of the generated games**, you could try the following pattern.

```
newgame --template cat
newgame --template fish
```

All template bit-mask patterns are stored in `data/templates.py`, you could add new templates into the `TEMPLATE_DICT` Python dictionary, and then it could be used as the template by specifying the key name of the dictionary when using `newgame` command.

Notice that **not every bit-mask is a valid bit-mask of Sudoku game**, and this program will throw error if no valid games could be generated from the given template after a maximum amount of times of attempt.

Output Redirection

Thanks to `cmd2`, we are allowed to redirect the output of nearly any command from `stdout` into other destinations, for example, a file.

```
# this will output the result on terminal
show -c
# this will output the result into a file called `output.txt`
show -c > output.txt
```

Ensure UTF-8 Encoding

Notice that **there could be some encoding issues when writing into a non UTF-8 encoded file** since some of the command output contain UTF-8 characters.

To ensure the correct format, it's recommend to enable `utf8` mode with Python by setting the following environment variable to 1

```
PYTHONUTF8=1
```

You may need to restart your terminal after editing the environment variable. To check if the operation success, you could run the following python code:

```
import sys
print(sys.flags.utf8_mode)
```

If the output is 1, then the UTF-8 mode has been successfully enabled.

The [PEP 686](#) suggest UTF-8 be the default behaviour for Python, and we don't need to manually set the environment variable with Python version ≥ 3.15